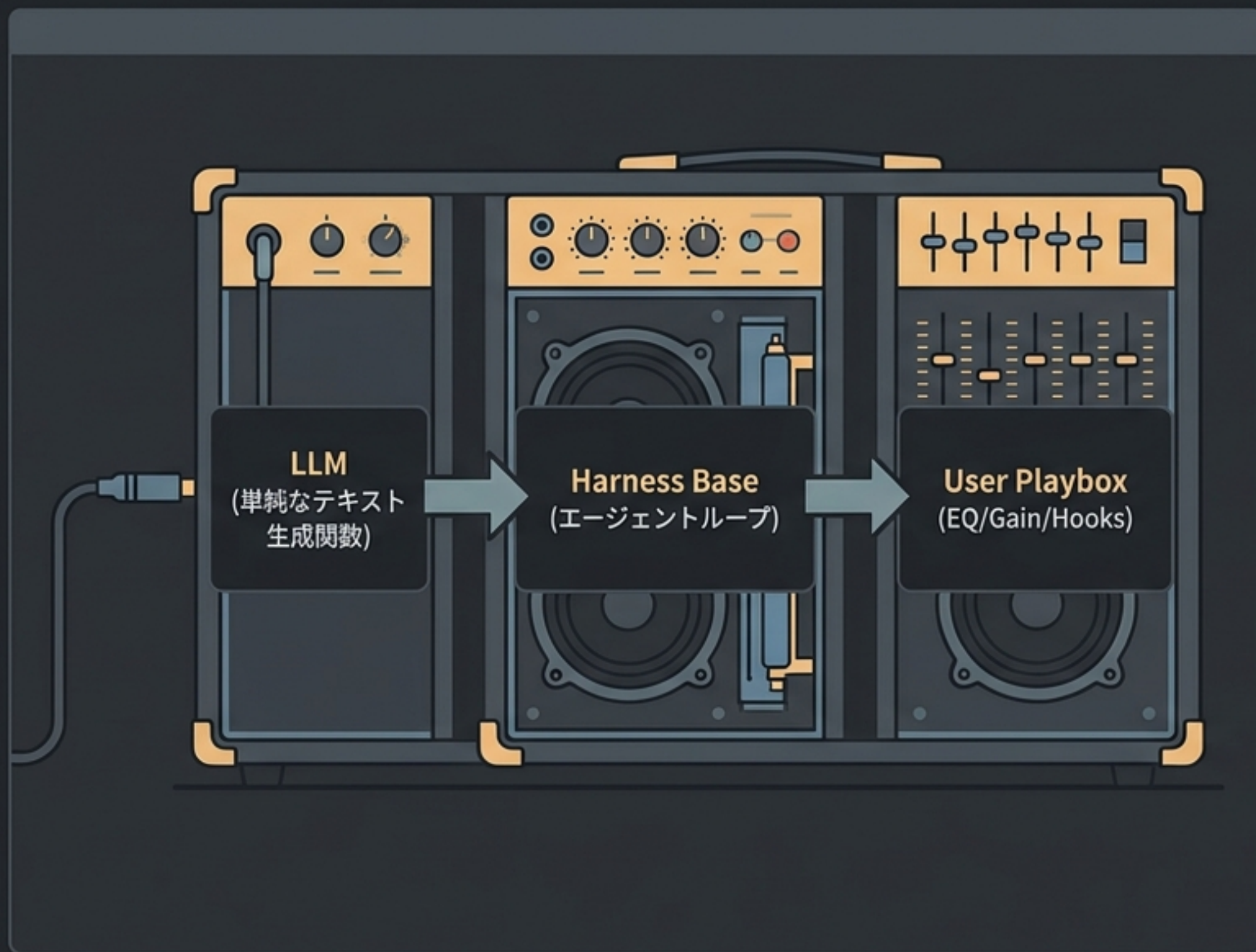


クロードコード ハーネスエンジニアリング完全指南

開発者のための制御盤：コンテキスト最適化から高度なセキュリティ設計まで

参照元：数理の弾丸 ⚡ 京大博士のAI解説 (船倉)

ハーネスエンジニアリングの哲学：ギターアンプのメタファー



ベースとなるLLMは単なる**テキスト生成器**。
そこに「**ハーネス**」を被せることで**自律的なAIエージェント**として機能する。

ギターアンプと同様、誰にでも適用できる
「**1つの正解（最強の設定）**」は存在しない。

自身のプロジェクトの課題、制約、文化に合わせて「**つまみ（設定）**」を調整し、**最適なアウトプット**を導き出す知識と経験が求められる。

アーキテクチャ：固定された基盤とカスタマイズ領域



固定された基盤 (Locked Platform)

- エージェントループそのもの
- 標準搭載のツール群
- コンテキストの積み上がり方の基本ルール

※ ユーザーは基本触れない領域。

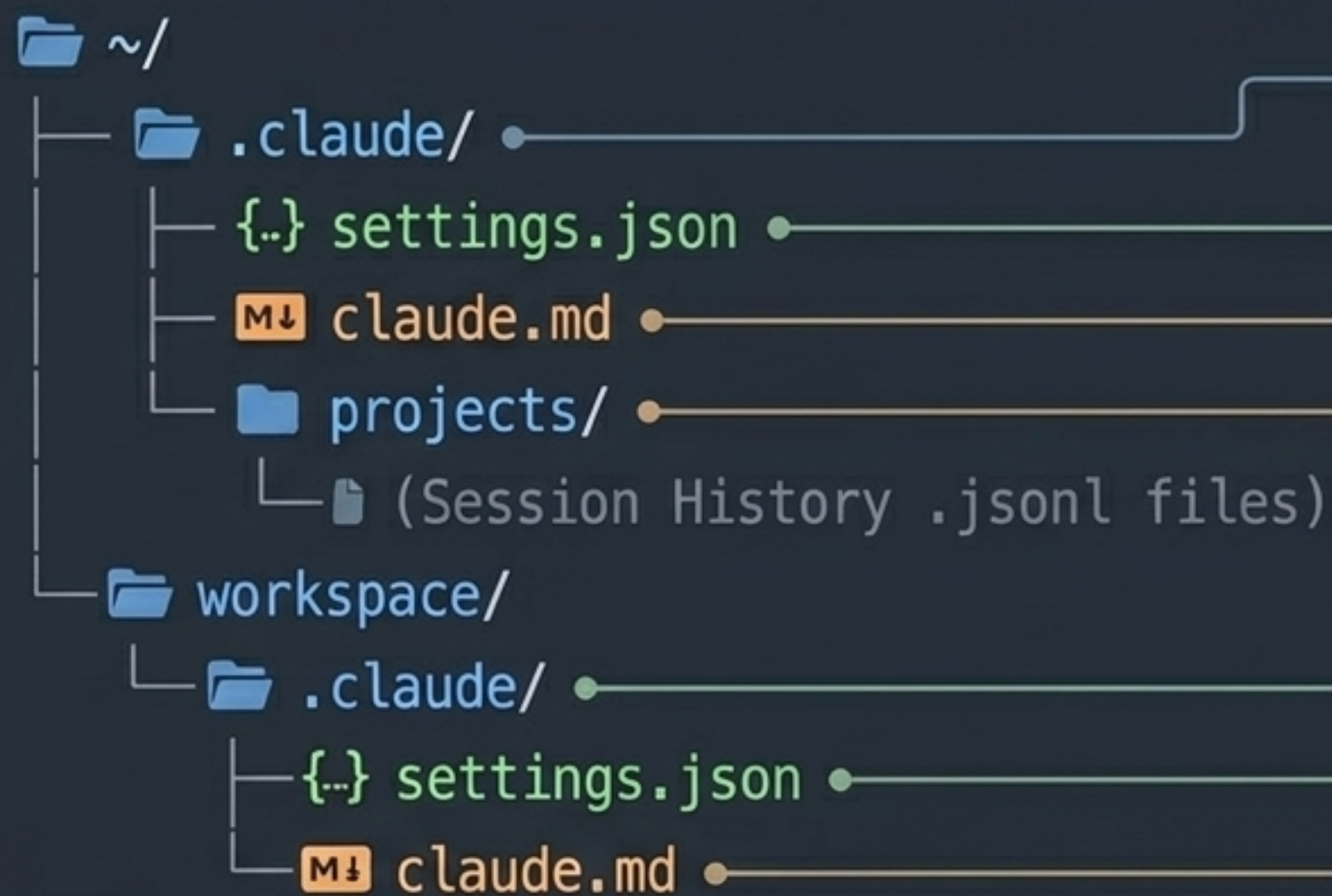


ユーザーカスタマイズ領域 (User Playbox)

- 使用モデルの選択
- Hook（特定のタイミングでのプログラム実行）
- ツール使用の権限管理（Permissions）
- ネットワークやファイルへのアクセス制御

※ ハーネスエンジニアリングの主戦場。

設定の物理的配置：.claude ディレクトリマップ



Global: ホーム直下。どのプロジェクトを開いても反映される全体設定。

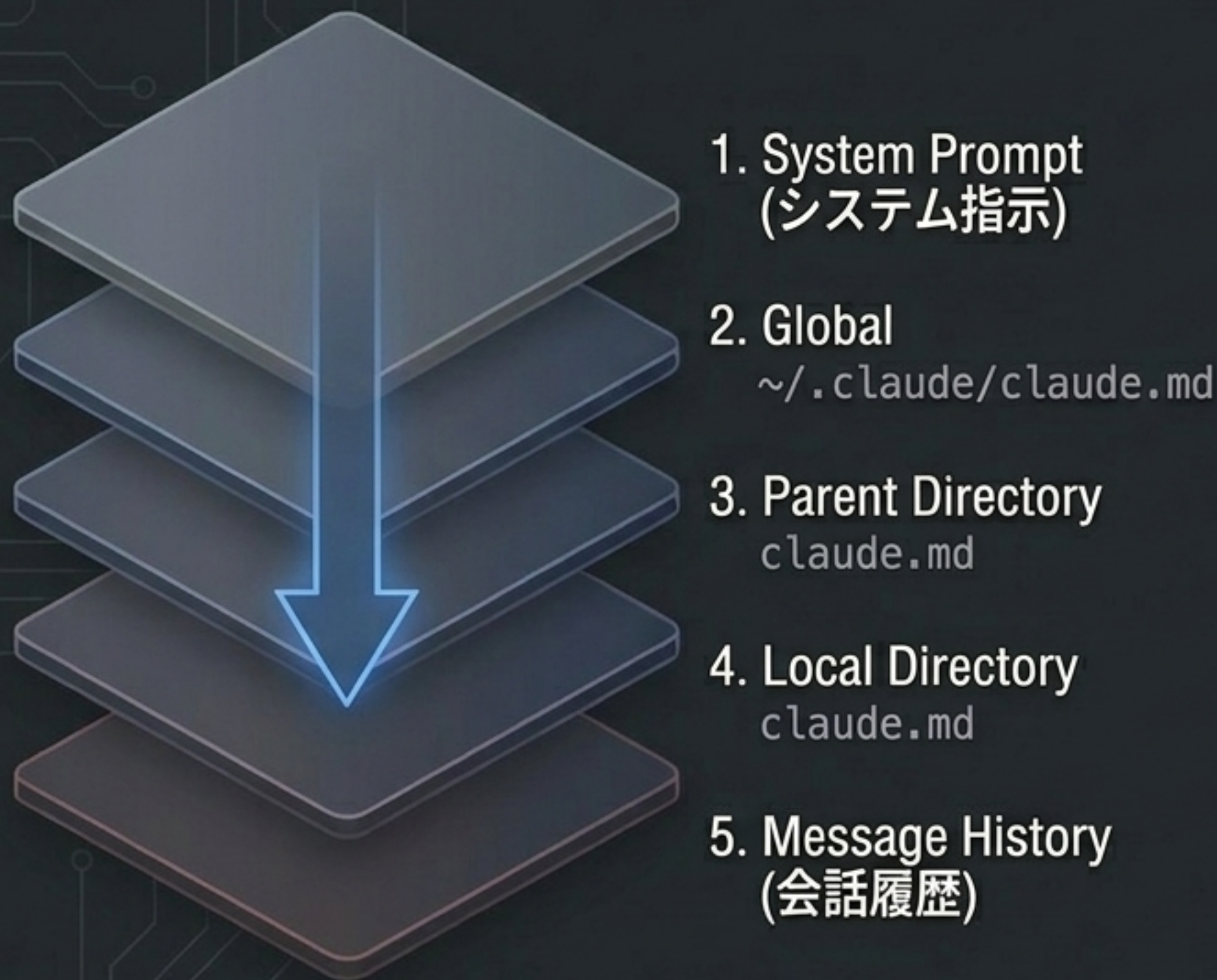
Local: 作業ディレクトリ直下。プロジェクト固有の個別設定（Globalを上書き可能）。

settings.json: ハーネスエンジニアリングの要（Hook、Sandbox、権限設定の心臓部）。

claude.md: プロンプトとルールの共有メモリ。

projects/: セッション履歴（.jsonl）の保存場所。

コンテキストのスタック構造：記憶はどのように構築されるか



⚠ **重要ルール:** 兄弟関係（横並び）のディレクトリや、子ディレクトリの `claude.md` は自動では読み込まれない。

Tip: `context` コマンドで現在読み込まれているファイル（Memory Files）を正確に確認可能。

```
> claude context
Memory Files:
- /Users/user/.claude/claude.md
- /workspace/.claude/claude.md
```


I/Oエンジニアリング：良質で安全な入出力の制御

Module 1: Hooks

特定条件での確実なプログラム実行（決定論的制御）。

Module 2: Compaction (圧縮)

メッセージ履歴の最適化とパフォーマンス維持。

Module 3: Branch (分岐/移行)

セッションの複製とコンテキストの持ち運び。

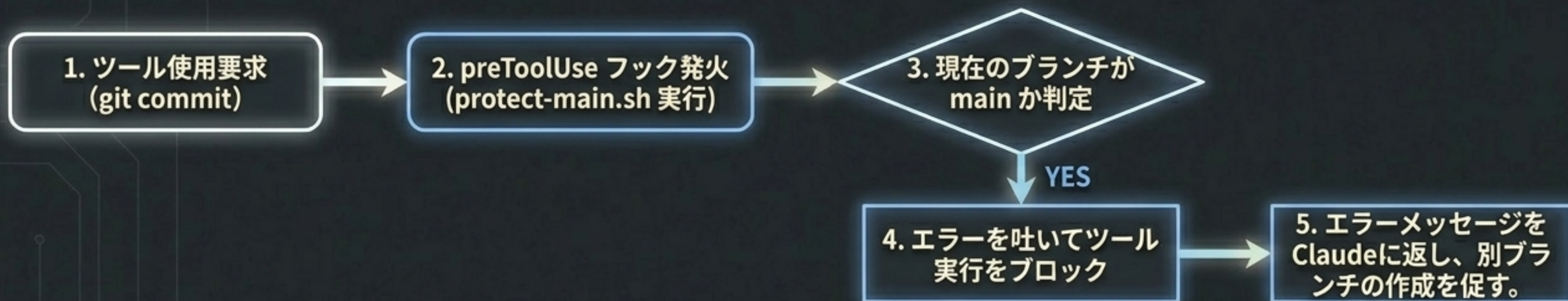
Hooks : ライフサイクルへの決定論的介入



Insight: LLMに「プロンプトでお願いする」のではなく、システムとして「強制執行」するための強力な手段。

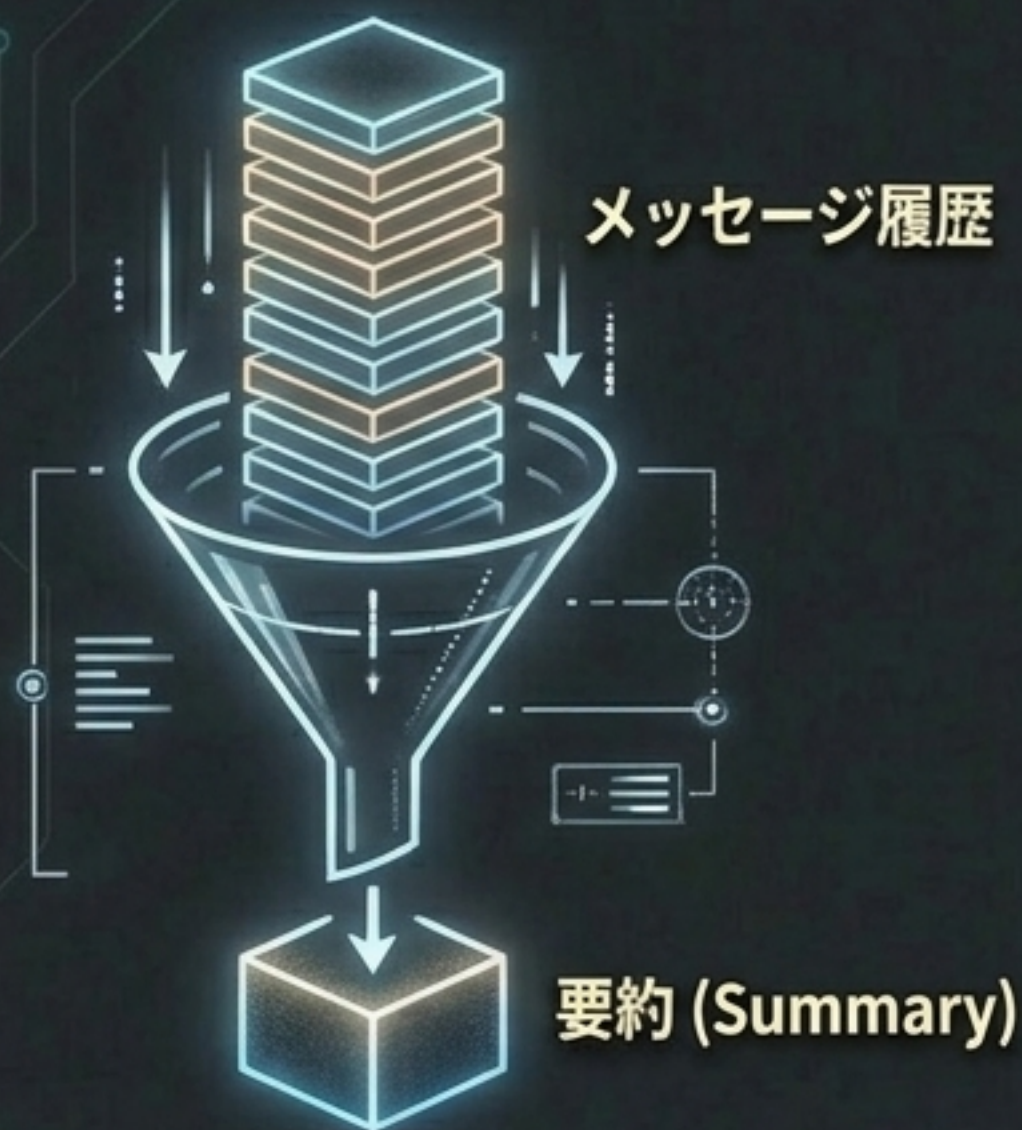
Hook実践：main ブランチの直接コミットを阻止する

```
settings.json ×
1 {
2   "hooks": {
3     "preToolUse": {
4       "bash": "protect-main.sh"
5     }
6   }
}
```



Insight: PR（プルリクエスト）運用をLLMにも厳格に守らせる実践的テクニック。

Compaction (圧縮) : コンテキストウィンドウの管理



コンセプト: メッセージ履歴が一定の閾値を超えると自動で要約され、元の詳細な対話ストロークは欠落する。

The 60% Rule



- デフォルト閾値: ウィンドウの約83.5%
- 推奨設定: `claudeAutoCompactPercentageOverride` を 60% に設定
- 理由: コンテキストが80%~90%まで埋まるとLLMのパフォーマンスが顕著に落ちるため、早めに圧縮させる

Pro-Tip: 圧縮によって失われたくない指示（人格設定など）は、`postCompact` フックを使って圧縮後に再度プロンプトとして注入できる。

Branch応用：作業ディレクトリをまたぐコンテキストの移行



課題: 標準の branch コマンドはセッションを複製できるが、「同じディレクトリ内」に限られる。

The Hack (Context Migration)

1. リサーチ完了

ディレクトリA（調査用）で有益な対話コンテキストを構築。



2. ファイル特定

~/ .claude/projects/ 内から、ディレクトリAのパスが名前になった .jsonl 履歴ファイルを見つける。



3. コピー

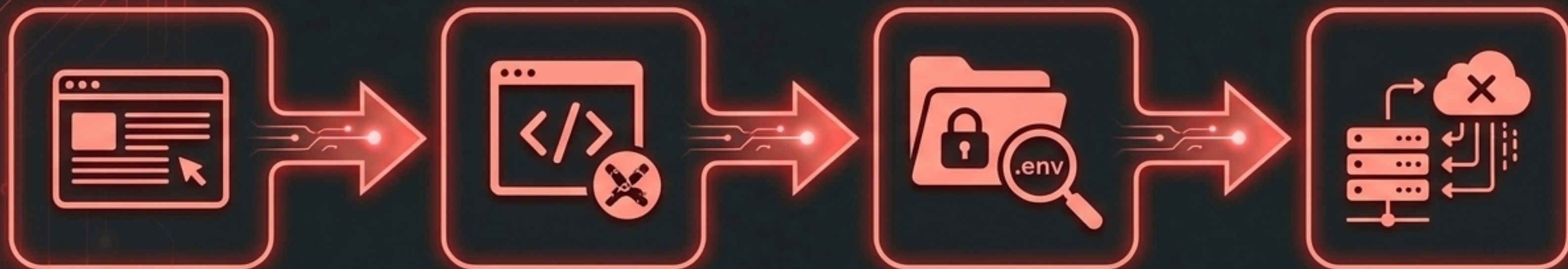
その .jsonl を、ディレクトリB（実装用）のハッシュフォルダに直接コピー。



4. 再開

ディレクトリBで特定のセッション再開コマンドを叩き、全く同じ記憶を持った状態で実装をスタート。

セキュリティの脅威モデル：プロンプトインジェクション



1. ClaudeがWeb検索ツールで悪意あるサイトを読み込む。

2. サイト内の隠しプロンプトがClaudeの指示を汚染（プロンプトインジェクション）。

3. Claudeが不正なスクリプトを実行し、ローカルの機密ファイル（.env 等）を読み取る。

4. 外部サーバーへ機密データを送信・漏洩。



注意: .env ファイル自体が悪なのではない。開発用プログラムが環境変数（APIキー等）を読み込むために必須のファイルである。だからこそ「LLMには読ませない」厳格な制御が必要。

Permissions設定：ツールアクセス権限の厳格化

1. **Deny (禁止)**：最初に評価される。絶対不可。

2. **Ask (確認)**：ユーザーに許可を求める。

3. **Allow (許可)**：自由に実行可能。

実践的防御



- read ツールの対象として .env を Deny に設定する。
- 効果：プログラマー（実行されるコード自体）は、.env を読めるが、Claude コード（エージェント）は read ツールを使って .env の中身を覗き見るができなくなる。

環境分離：標準サンドボックス vs Dockerアーキテクチャ

Native Sandbox (クロードコード標準)



コンセプト：
ファイルシステムとネットワークアクセスを分離。

現在の課題（バグ）：

- ⚠️ • 読み取り許可データの絞り込み設定が効かない
- ⚠️ • アクセス許可ドメインの絞り込みが効かない
- ⚠️ • Linux環境での隔離ツールへの引数渡しが不正

Docker Devcontainers (推奨手法)



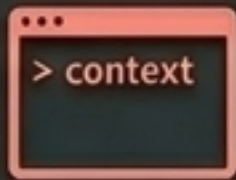
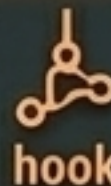
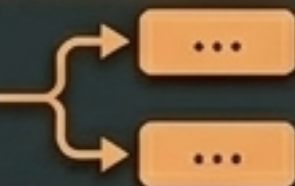
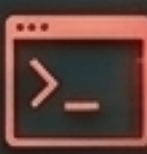
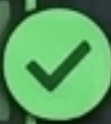
コンセプト：
コンテナによる完全な環境隔離（無料）。

メリット：

- ✓ • Volume Mount機能で「作業に必要なディレクトリ」だけを同期。
- ✓ • ネットワークもファイアウォールで制御可能。

結論：特に社用PCで機密性の高い作業を行う場合は、Dockerによるサンドボックス化を強く推奨。

The Harness Engineer's Checklist

- | | | | | |
|-----|----|----------------------|--|---|
| [1] | ON | STATUS:
ACTIVE | コンテキストの把握: context コマンドで意図した .md がスタックされているか確認する。 |   |
| [2] | ON | STATUS:
SECURE | 決定論的制御: preToolUse Hookを活用し、Linter実行やmainブランチ保護を自動化する。 |   |
| [3] | ON | STATUS:
OPTIMIZED | 圧縮の最適化: AutoCompactPercentage を60%に設定し、パフォーマンス低下を防ぐ。 |   |
| [4] | ON | STATUS:
DENIED | 権限の壁: .env ファイルへの read ツールアクセスを Deny に設定する。 |    |
| [5] | ON | STATUS:
ISOLATED | 環境の隔離: 社用PCでは標準サンドボックスを過信せず、Dockerで堅牢な分離環境を構築する。 |   |

「最強の設定」は存在しない。自身の環境に合わせて最適化を。

プレゼン内容出典：数理の弾丸 ⚡ 京大博: NotebookLM